Coding Standards

Version: 1.0

Purpose

This document defines the coding standards for Trading Platform Pro and its primary application, the Trading Cockpit.
The objective is to maintain a readable, deterministic, strongly typed and testable codebase.
Trading-critical code requires particular care because technical side effects may affect external broker state.
Code shall preserve the documented architecture boundaries and explicit workflow ownership.

General Principles

Every line of code should be:
• readable
• maintainable
• testable
• deterministic
• explicit
Code is written for humans first.
Prefer simple and explicit implementations over clever abstractions.
Avoid introducing generic infrastructure for hypothetical future requirements.

Python Version

Mandatory:
• Python 3.13
The supported Python version shall remain synchronized with:
• dependency definitions
• CI configuration
• development documentation

File Encoding

Use:
• UTF-8
• Unix line endings (LF)
Repository formatting rules shall remain consistent across development environments.

Required Imports

Every Python module should begin with:
```python
from __future__ import annotations
```
unless a documented technical reason prevents its use.

Type Hints

Type hints are mandatory for:
• function parameters
• return values

• public attributes
• public interfaces
• ports
• repository interfaces
• application service interfaces
Prefer explicit types over `Any`.
Avoid generic dictionaries as stable application contracts.
Avoid:
```python
def submit_order(data: dict) -> dict:
...
```
Prefer:
```python
async def submit_order(
request: SubmitOrderRequest,
) -> BrokerOrderSubmission:
...
```

Type Ownership
Types shall remain owned by the correct architecture layer.
Domain owns:
• Entities
• Value Objects
• Domain Events
• Domain Exceptions
• Domain identities
Application may own:
• Commands
• Queries
• Application DTOs
• application-oriented result types
Infrastructure owns:
• provider models
• persistence models
• transport models
• framework-specific models
Presentation owns:
• widget state
• presentation models
• UI-specific types
Types shall not be moved between layers solely to avoid translation code.

Domain Types
Domain types shall remain technology-independent.
Examples:
```python
InstrumentId
OrderId
PositionId
Money
Price
Quantity
```

```
Domain types shall not depend on:
• SQLAlchemy
• PySide6
• broker SDKs
• provider payloads
• transport frameworks
Use explicit Domain types when primitive values have business meaning.
Avoid primitive obsession in business-critical workflows.
---
# Commands and Queries
Application operations should use explicit Commands or Queries where they clarify workflow intent.
Example:
```python
@dataclass(frozen=True)
class CreateOrderCommand:
decision_id: TradingDecisionId
instrument_id: InstrumentId
action: OrderAction
quantity: Quantity
```
Commands represent requested state-changing operations.
Queries represent read-oriented requests.
Do not use one generic request model for unrelated workflows.

Application DTOs
Application DTOs may transfer structured data across application boundaries.
DTOs shall:
• use explicit fields
• be strongly typed
• remain free from provider-specific behaviour
• represent application requirements
DTOs are not automatically Domain Entities.
Avoid adding Domain behaviour to transport-oriented DTOs.

Provider Models
Provider-specific models belong to Infrastructure.
Examples:
```python
IbContract
IbOrder
ProviderQuoteMessage
```
Provider models shall not cross into Domain APIs.
Adapters shall translate provider models into application-oriented or domain-oriented types.
Provider-specific field names shall remain inside provider integration boundaries where practical.

Persistence Models
Persistence models belong to Infrastructure.
Examples:
```python
```
```

OrderRecord
PositionRecord
SqlAlchemyOrderModel
```

Persistence models shall not be used as Domain Entities.
Repository interfaces shall not expose SQLAlchemy models.
Translation between persistence and Domain models shall remain explicit.
---
# Naming

Names shall communicate responsibility and business meaning.
Avoid vague names.
---
# Modules

Use:
```text
snake_case
```

Example:
```text
market_data_service.py
```

Module names shall describe the owned capability.
Avoid generic modules such as:
```text
helpers.py
utils.py
common.py
```

when a more specific capability name is available.

Classes

Use:
```text
PascalCase
```

Examples:
```python
OrderApplicationService
TradingCandidateService
InteractiveBrokersOrderAdapter
```

Avoid vague class names such as:
```python
Manager
Handler
Processor
Helper
```

unless the complete name communicates one specific responsibility.

Functions and Methods
```

Use:
```text
snake_case
```
Examples:
```python
calculate_position_size()
submit_order()
load_open_positions()
```
Function names shall describe one operation.
Avoid generic names such as:
```python
process()
handle()
execute()
```
without capability context.

Variables
Use:
```text
snake_case
```
Example:
```python
market_price
```
Variable names shall communicate meaning.
Avoid unnecessary abbreviations.

Constants
Use:
```text
UPPER_CASE
```
Example:
```python
DEFAULT_TIMEOUT
```
Business values shall not be hidden as unexplained constants in technical modules.
Business-controlled parameters require explicit capability ownership.

Functions
Functions should:
• have one responsibility
• remain focused
• be easy to test
• make side effects explicit
• use meaningful return types
Avoid unnecessary side effects.
A function that performs an external side effect shall communicate that responsibility through its owning workflow and name.

Classes

Classes should:
• represent one concept
• have a single responsibility
• expose a clear API
• hide implementation details
• preserve architecture ownership
Avoid large service classes containing unrelated workflows.
Prefer capability-oriented classes.

Imports

Import order:
1. Standard Library
2. Third-party Libraries
3. Project Imports
Avoid wildcard imports.
Imports shall preserve architecture dependency direction.
A technically valid Python import may still violate the documented architecture.

Architecture Dependencies

Always preserve the documented dependency direction.
```text
Presentation
↓
Application
↓
Domain
Infrastructure
↓
Application / Domain Ports
```
Domain shall remain independent from:
• Infrastructure
• Presentation
• frameworks
• external services
Application shall not depend on Presentation.
Presentation shall not access concrete broker or market data adapters directly.

Dependency Injection

Dependencies shall be injected rather than created inside business logic.
Avoid:
• global service instances
• hidden dependencies
• service locators inside Domain logic
• direct adapter construction inside Application workflows
Prefer constructor injection for stable dependencies.
Example:
```python
class OrderApplicationService:
```

```
def __init__(
self,
order_repository: OrderRepository,
broker_order_port: BrokerOrderPort,
) -> None:
self._order_repository = order_repository
self._broker_order_port = broker_order_port
```

Dependencies shall remain explicit.

Domain-Driven Design

Business rules belong to Domain.
Application coordinates use cases and workflows.
Infrastructure provides technical implementations.
Presentation contains no business logic.
Do not place business rules in:

- widgets
- SQLAlchemy models
- broker adapters
- monitoring collectors
- logging handlers

State Transitions

Business-critical state transitions shall be explicit and deterministic.
Prefer:

```python
order.acknowledge()
```

over:

```python
order.status = "acknowledged"
```

where Domain behaviour owns the transition.
State transition methods shall:

- validate current state
- reject invalid transitions
- preserve invariants
- produce deterministic resulting state

Avoid unrestricted public mutation of business-critical state.

State Representation

Use explicit state types.
Prefer:

```python
OrderStatus.ACKNOWLEDGED
```

Avoid:

```python
"ack"
"ok"
"done"
```

String-based state shall not be used where a stable state model exists.

Financial Values

Financial values require explicit semantics.
Use `Decimal` where decimal financial precision is required.
Avoid uncontrolled binary floating-point arithmetic for:
• money
• prices
• fees
• persisted financial values
Example:
```python
price = Decimal("123.45")
```
Do not construct financial `Decimal` values from uncontrolled binary floats.
Avoid:
```python
Decimal(123.45)
```
Prefer:
```python
Decimal("123.45")
```
or an explicitly normalized provider conversion.

Unavailable Financial Data

Unavailable financial data shall remain explicit.
Do not convert missing values silently to:
```python
0
0.0
Decimal("0")
```
when zero has valid business meaning.
Prefer:
```python
Price | None
```
or an explicit availability-aware result type.
Missing data and zero are different business states.

Quantities

Quantities shall use explicit types where business meaning requires them.
Do not assume all trading quantities are integers unless the instrument or workflow guarantees integer quantity.
Quantity validation belongs to the owning Domain or Application capability.
Provider quantity formats shall be translated at the adapter boundary.

Timestamps

Time-dependent code shall preserve timestamp semantics.
Where relevant distinguish:
• application timestamp

• broker timestamp
• market data source timestamp
Timezone context shall remain explicit.
Avoid ambiguous naive datetimes in trading-critical interfaces.
Use timezone-aware datetime values where cross-system time semantics matter.

Async Programming

Use `async` and `await` consistently.
Async code shall:
• never block the event loop
• avoid nested event loops
• use explicit timeouts where appropriate
• support controlled cancellation
• preserve exception context
• release owned resources
• remain focused
Do not mark functions async without an asynchronous boundary.

Blocking Operations

Blocking operations shall not execute directly on the AsyncIO event loop.
Examples:
• blocking file operations
• long CPU-bound calculations
• blocking third-party APIs
• synchronous waits
Blocking work shall be isolated through an approved execution mechanism.
The UI thread shall also remain free from long-running blocking operations.

Task Ownership

Long-running asynchronous tasks require explicit ownership.
Avoid unmanaged:
```python
asyncio.create_task(...)
```
for application background services.
A long-running task shall have:
• owner
• identity
• startup behaviour
• cancellation behaviour
• failure handling
• shutdown behaviour
Tasks shall be registered with the appropriate runtime lifecycle component.

Cancellation

Cancellation is part of normal async control.
Cancellation shall:
• remain distinguishable from success
• propagate where appropriate
• release resources
• preserve valid state

• not imply external acknowledgement
Do not swallow cancellation silently.
Cancellation handling shall remain explicit in business-critical workflows.

Timeouts

External operations shall use explicit timeout behaviour where appropriate.
Examples:
• broker communication
• market data requests
• external APIs
Timeout shall remain distinguishable from:
• rejection
• cancellation
• disconnection
• unavailable capability
Do not convert every timeout into a generic failure result.

Error Handling

Errors shall:
• fail explicitly
• preserve meaningful context
• use appropriate exception types or result states
• never be silently ignored
Avoid:
```python
try:
...
except Exception:
pass
```
Unexpected exceptions shall preserve technical context.
Expected business failures shall use explicit business states or Domain Exceptions where appropriate.

Exception Ownership

Exceptions shall be translated at architecture boundaries.
Examples:
```text
Provider Error
↓
Infrastructure Adapter
↓
Application-Oriented Integration Error
```
Provider-specific exceptions shall not leak into Domain workflows unless explicitly required.
Infrastructure failures shall not be disguised as Domain rule violations.

Logging

Use the project logging infrastructure.
Never use:
```python
```

print(...)
```
for production diagnostics.
Use deterministic structured event names.
Example:
```python
logger.info(
"ORDER_ACKNOWLEDGED",
extra={
"order_id": str(order_id),
"environment": environment.value,
},
)
```
Logging shall not become the authoritative business state.

Logging Context

Business-critical logs should include relevant canonical identity.
Examples:
• `instrument_id`
• `candidate_id`
• `decision_id`
• `order_id`
• `position_id`
Trading-related logs shall include PAPER or LIVE context where relevant.
Do not log complete provider payloads blindly.

Sensitive Data

Never log:
• passwords
• tokens
• API keys
• broker credentials
• secret environment variables
• private keys
Personal and account information shall be omitted or masked where practical.
Sensitive values shall not appear in exception messages.

Comments

Prefer self-explanatory code.
Comments should explain:
• why
• architectural constraints
• non-obvious external behaviour
• business-critical reasoning
Comments should not merely restate what the code does.
Avoid:
```python
# Increment counter

counter += 1
```
```

Useful example:
```python
# Broker reconnection must not repeat order submission because

# the previous transmission state may be unknown.
```

Docstrings

Public modules, classes and functions should contain meaningful docstrings where the contract is not obvious.
Docstrings should explain:
• purpose
• important behaviour
• failure semantics
• side effects where relevant
Do not duplicate type hints in verbose parameter descriptions without additional value.

Formatting

Formatting is enforced using:
• Ruff
• Ruff Format
CI verification uses:
```bash
ruff check .
ruff format --check .
```
Do not manually fight the formatter.

Order Side Effects

Order submission is an external business-critical side effect.
Code that may submit an order shall remain explicitly identifiable.
Order submission shall occur only through an Application-owned workflow.
Avoid direct broker submission from:
• Presentation
• Domain
• monitoring
• logging
• health checks
• reconciliation observation
Adapters execute technical submission.
Application workflows decide whether submission is requested.

Order Submission State

Code shall preserve the distinction between:
```text
Submission Requested
Transmitted
Acknowledged
```

Rejected
Partially Filled
Filled
```
Do not use one boolean such as:
```python
submitted = True
```
to represent multiple lifecycle states.
A successful adapter call shall not automatically mean broker acknowledgement.

Duplicate Submission Risk

Duplicate order submission risk shall be considered explicitly.
Order submission workflows shall use stable identities where required.
Example:
```python
OrderId
OrderSubmissionKey
```
Duplicate-prevention rules belong to Application workflows.
Adapters shall not independently decide that repeated submission is safe.
Code shall not blindly repeat order submission after:
• timeout
• disconnect
• reconnect
• unknown acknowledgement state
Unknown external state requires explicit workflow handling.

Retry Behaviour

Retries shall be explicit.
Before implementing retry evaluate:
• idempotency
• external side effects
• duplicate execution risk
• provider behaviour
• workflow ownership
Automatic retry may be appropriate for selected read operations.
Automatic order submission retry is prohibited unless explicitly defined and approved by the owning order workflow.
Avoid generic retry decorators around trading-critical side-effect operations.

Broker Code

Broker adapter code shall focus on:
• provider communication
• provider model translation
• provider identifier translation
• provider error translation
• technical connection lifecycle
Broker adapters shall not contain:
• candidate acceptance rules
• trading decisions
• portfolio risk decisions
• duplicate submission policy
```

• position close decisions
Provider-specific logic shall remain isolated.

Market Data Code

Market data adapter code shall focus on:
• provider communication
• quote translation
• timestamp preservation
• subscription lifecycle
• provider error translation
Market data adapters shall not contain trading decision rules.
Cached values shall not silently appear as current provider values.
Freshness state shall remain explicit where required.

Repository Code

Repositories implement persistence requirements.
Repository code shall:
• preserve Domain identity
• translate persistence models explicitly
• use controlled transactions
• expose meaningful persistence operations
Repositories shall not contain trading decisions.
Avoid generic repositories created solely to reduce repeated method names.

Presentation Code

Presentation code shall focus on:
• rendering
• user interaction
• presentation state
• application command invocation
Widgets shall not:
• construct broker adapters
• access SQLAlchemy sessions
• submit broker orders directly
• implement trading rules
User trading actions shall call explicit Application workflows.

Monitoring Code

Monitoring code observes technical and operational state.
Monitoring code shall not:
• submit orders
• cancel orders
• reconnect brokers independently
• repair Domain state
• resolve reconciliation discrepancies
Recovery actions require explicit runtime or application ownership.

Configuration Management

Configuration shall:
• be externalized

• be strongly typed
• be validated
• use deterministic precedence
• never contain secrets in source-controlled files
Configuration compatibility shall be evaluated based on real consumers.
Do not preserve obsolete internal configuration solely for hypothetical backward compatibility.
Unused configuration shall be removed.

Security Standards

Never commit:
• credentials
• secrets
• certificates
• private keys
Use approved environment variables, operating system credential storage or secret providers.
Secret access shall remain explicit.
Do not expose secrets through:
• logs
• exceptions
• UI diagnostics
• generated documentation

Documentation Standards

Documentation shall remain synchronized with implementation.
Markdown under:
```text
docs/
```
is the documentation source of truth.
Generated DOCX and PDF files are derived artifacts.
Generated documentation shall not be edited manually.
Code changes affecting documented contracts shall update the relevant Markdown source.

Testing

Every new feature should include appropriate tests.
Bug fixes should include regression tests whenever practical.
Trading-critical code requires scenario-focused regression tests.
Tests shall verify where relevant:
• state transitions
• unavailable data
• timeout behaviour
• cancellation
• duplicate submission protection
• provider translation
• PAPER and LIVE separation
• reconciliation behaviour
Tests shall not require LIVE trading.

Deterministic Tests

Tests shall avoid uncontrolled dependencies on:
• wall-clock time

• network state
• LIVE broker services
• random data
• developer-local configuration
Use:
• controlled clocks
• fake adapters
• deterministic fixtures
• explicit test configuration
when required.

Performance Guidelines

Optimize only after measurement.
Prefer:
• simple algorithms
• readable code
• predictable performance
Avoid premature optimization.
Trading Cockpit performance-sensitive areas may include:
• UI responsiveness
• AsyncIO event loop latency
• market data update paths
• structured logging volume
Performance optimization shall not weaken business correctness.

Code Smells

Avoid:
• duplicated code
• long methods
• deep nesting
• hidden side effects
• large classes
• magic numbers
• unused code
• generic dictionaries as stable contracts
• provider models outside Infrastructure
• persistence models outside Infrastructure
• unmanaged background tasks
• broad exception swallowing
• generic retry around order submission
• boolean state representing complex lifecycle
• silent conversion of unavailable financial data to zero

Code Review Checklist

Before approving code verify:
• responsibility is clear
• naming is explicit
• typing is complete
• architecture boundaries are preserved
• Domain types remain technology-independent
• provider models remain in Infrastructure
• persistence models remain in Infrastructure
• side effects are explicit

• state transitions are deterministic
• unavailable financial data remains explicit
• timestamp semantics are clear
• async code does not block the event loop
• long-running tasks have ownership
• cancellation is handled
• timeout behaviour is explicit
• order submission lifecycle remains precise
• duplicate submission risk is evaluated
• retry behaviour is explicit
• PAPER and LIVE impact is evaluated
• sensitive data is protected
• tests are sufficient
• documentation is synchronized

Related Documents

• Product_Vision.md
• Product_Roadmap.md
• Project_Overview.md
• Architecture.md
• Domain_Model.md
• Infrastructure.md
• Technical_Specifications.md
• API_Guidelines.md
• Configuration.md
• Runtime.md
• Logging.md
• Monitoring.md
• CI_CD.md
• Development_Guidelines.md
• Testing_Strategy.md
• AGENTS.md